1.

a. is true $f(n)$ and $g(n)$ functions $f(n) \in O(g(n))$ or $g(n) \in O(f(n))$

if $f(n) = n^2$ $\quad g(n) = n^3$ $\quad f(n) \in O(n^3)$ but $\cancel{n^3 \in O(n^2)}$

$n^3 \in \omega(n^2)$ is true

b. if $f(n) \in O(g(n))$, is $\frac{f(n)}{h(n)} \in O\left(\frac{g(n)}{h(n)}\right)$ true?

$$\frac{f(n)}{h(n)} \leq c_1 \cdot \frac{g(n)}{h(n)} \qquad 0 \leq \frac{f(n)}{h(n)} \leq c_1 \cdot \frac{g(n)}{h(n)}$$

if $h(n) \leq 0$ its not true, other wise its tre.

c. if $f(n) \in O(g(n))$ then the following true: $f(n)^k \in O(g(n)^k)$

$0 \leq f(n) \leq c_1 g(n)$ $\quad n \geq n_0$

if $k \geq 0$ $\quad f(n) = n$ $\quad g(n) = n^2$ $\quad k = 1, 2 \ldots +\infty$

$n^k \leq c \cdot n^{2k}$ true $\quad \underline{f(n)^k \in O(g(n)^k)}$

other wise: $f(n) = n$ $\quad g(n) = n^2$ $\quad k = -1, -2 \ldots -\infty$

$k = -1$ $\quad n^k \leq c \cdot n^{2k}$

$\frac{1}{n} \geq c_1 \cdot \frac{1}{2^k}$ so $\underline{f(n)^k \in \Omega(g(n)^k)}$

2.

a. $n^3 \in O(2^n)$

$n^3 \leq c_1 \cdot 2^n$ $\quad n \geq n_0$

$$\lim_{n \to \infty} \frac{n^3}{2^n} = \lim_{n \to \infty} \frac{3n^2}{2^n \ln 2} = \lim_{n \to \infty} \frac{6n}{2^n \ln 2^2} = \lim_{n \to \infty} \frac{6}{2^n \cdot \ln 2^2} = \frac{1}{\infty} = 0$$

so $\underline{n^3 \in O(2^n)}$

2.
b.

$2^n \in o(3^n)$ little $o = f(n) < c.g(n)$

$2^n < c_1 . 3^n$ $n \geq n_0 \geq 0$ $c > 0$

$c = 1$ $2^n < 3^n$
$c_1 = 2$ $2^n < 2.3^n$

$c = k$ $2^n < \underline{k.3^n}$ its true ✓
$c = k+1$ $2^n < (k+1).3^n = 2^n < 3^n + \underline{k.3^n}$

c. $n! \in O(100^n)$

$\underline{\text{Stirlings formula}}$: $n! \cong \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$ so: $\underline{n! \in \omega(100^n)}$

$$\lim_{n \to \infty} \frac{n!}{100^n} = \lim_{n \to \infty} \sqrt{2\pi n} . \left(\frac{n}{100e}\right)^n = \infty$$

constant

3. pseudocode

```
function ISR (A[0 ... n-1], item)

    for P <- 0 to n-1 do
        if A[P] = item then
            return P
        end if
    end for
    return -1
end
```

Best Case: find in first element $\underline{ISR(n) \in \Theta(1)}$

Worst case: item found at last element or not found in list

$\underline{ISR(n) \in \Theta(n)}$

3.

average case

$$E(i) = \sum_{i=1}^{w(n)} i \cdot P_i$$

P_i = probability that x occur in ith pos.

x in list

$x = L[i] = \frac{P \cdot a}{n}$

x not in list

$\frac{P \cdot a}{n} - (1-P)$

$$A(n) = \sum_{i=1}^{w(n)} i \cdot P_i$$

$$= \sum_{i=1}^{n-1} i \cdot \frac{Px}{n} + n \cdot \left(\frac{Px}{n} + (1-P) \right)$$ => found + not found

$$= \frac{Px}{\cancel{n}} \cdot \frac{\cancel{n} \cdot n-1}{2} + Px + n - nP$$

$$= (Px \cdot n-1 + 2Px + 2n - 2nP)/2$$

$$= (Pxn + Px + 2n - 2nP)/2$$

$$= n\left(\frac{Px + 2 - 2P}{2}\right) + \frac{Px}{2} \quad \in \Theta(n)$$

4. Algorithm Enigma (A[0..n-1][0..n-1])

```
for i=0 to n-2 do
    for j=i+1 to n-1 do
line 4 → if A[i,j] != A[j,i]
            return false
return true.
```

a. Problem Size = n.n

b. main operation is comparison at line 4.

c. calculate whether matrix is symetric.

4.

Best case if first comparison fail

$B(n) \in \Theta(1)$

Worst case

$$W(n) = \sum_{i=0}^{n-2} \sum_{J=i+1}^{n-1} = \sum_{i=0}^{n-2} n-i-1 = (n-1)+(n-2)+ \cdots +1$$

$$W(n) = \frac{(n-1)(n)}{2} \Rightarrow W(n) \in \Theta(n^2)$$

5.

```
Function myFun2(A[1...n])
    if n <= 1
        return 4
    else return myFun2(A[1.. n/3]) + myFun2([A[2n/3....n])
```

a. Problem size = n, size of array

b.

```
Function myFun3(A[1...n])
    if n <= 2 // if size less then 3 ->  ↓ + ↓  <- min requirement
        return 0;
    return 1 + myFun3(A[1..n/3]) + (myFun3(A[2n/3 -- n])
```

c. subsitute $n = x^k$ $x \Rightarrow 3$ because program takes $\frac{1}{3}$ of array for new calls

$\underline{\underline{n = 3^k}}$

in every call, there are 2 addition

total number of additions $\Rightarrow \underline{\underline{2 \cdot 3^k}}$

d. myFun2 $\rightarrow n = 3^k$

$k = \log_3 n$ $f(n) \in O(\log_3 n)$

myFun2 $\rightarrow f_1(n) \in O(\log_3 n)$

7.

pseudocode for # of 0 in array

```
function findZeroNum (arr[0...n-1], statIndex, endIndex)
    if statIndex > endIndex
        return 0
    mid = (statIndex + endIndex)/2
    res = findZeroNum(arr, statIndex, mid-1) + findZeroNum(arr, mid+1, endIndex)
    if arr[mid] = 0
        return 1+ result
    return result
end
```

Best - Worst case: The algorithm will search in all array for best and worst case.

$B(n) = \Theta(\log n)$ Worst $= \Theta(\log n)$

$B(n) = W(n)$ So, Average case $\Theta(\log n)$

10.

Master Theorem

$T(n) = a.\, T(\frac{n}{b}) + f(n)$

$\varepsilon > 0$

$c < 1$

$$T(n) \in \begin{cases} \Theta(n^{\log_b a}) & f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)), & f(n) = \Omega(n^{\log_b a + \varepsilon}) \text{ and } af(n/b) < cf(n) \end{cases}$$

a) $f(n) = 3f(\frac{n}{2}) + n^2$, $f(1) = 4$

$g(n) \in \Omega(n^{\log_2 3 + \varepsilon})$ case 3

$b = 2$
$a = 3$

so $n^{\log_b a} = n^{\log_2 3}$

$f(n) \in \Theta(n^2)$

b) $f(n) = 3f(n/2) + n^2 \log n$, $f(1) = 1$

$b = 2$
$a = 3$
$g(n) = n^2 \log n$

$n^2 \log n \in \Omega(n^{\log_2 3 + \varepsilon})$

$T(n) \in \Theta(f(n)$

$T(n) \in \Theta(n^2 \log n)$